

These notes covers all the control structures in the python language and possible all modern programming languages today.

Control Structures

Control structures are the building blocks that determine the flow of execution of a program. They let you decide *what happens, when it happens and how often it happens*.

By understanding control structures, someone is able to predict the effects of a set of code statements.

There are three control structures (the first three), however some software practioners also recognizes that there is one other one (function).

Each structure some advantage.

Contents

1. [Sequence](#)
2. [Branching](#): program is able to fulfill more than one task.
3. [Looping](#): less coding require
 - while
 - for
 - A. using the range object
 - B. using an enumerable
 - C. using the enumerate function
4. [Functions](#): allows code re-use

Sequence

Sequence is the most basic control structure.
It represents the natural order of processing

- You start with the first statement and then go to the next
- This continues until you get to the end.
- There are no skips, no jumps and no repeats.
- So if there are ten statements, then each is processed sequentially.

Because the same statements always execute in the same order, sequence is deterministic.
i.e. it will always produce the same effect.

Example

If the intended purpose of a set of statements is to sum three numbers, then that is the only task it and will do!

statement1

↓

statement2

↓

statement3

↓

statement4

↓

statement5

Branching/Condition/Selection

These allows your program to make decisions and execute different code paths based on conditions

if statement

```
if «condition»:
```

```
    statement1
```

```
    statement2
```

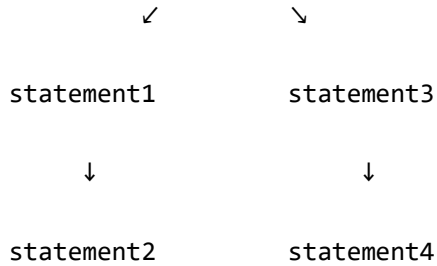
```
else:
```

```
    statement3
```

```
    startment4
```

N.B. either statements 1 and 2 are processed or statement 3 and 4. Both sets are not ignore or both set are not processed.

«condition»



match statement

```

switch «variable/expression»:

    case «label1»:

        statement1

    case «label2»:

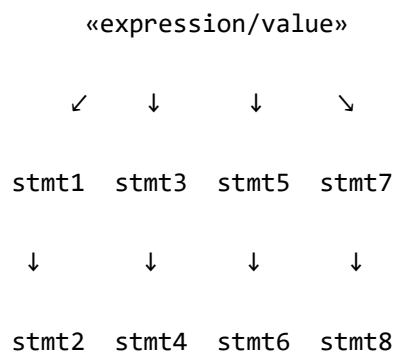
        statement2

    case _:

        statement3

```

This is different from the previous if-statement, because branching is not dichotmous, but instead multi-way.



The next notebooks covers this control structure in more details

Repetition/Iteration/Looping

These control structures provide a way to process/execute a statement or block of statements 0 or more times

The `while` statements repeats the block of code until a condition fails

```
while «test_condition»:  
  
    statement1  
  
    statement2
```

Statements 1 and 2 are processed until the condition fails

The `for` statement repeats the code according to the properties of the iterable

```
for «iterator_variable» in «iterable»:  
  
    statement1  
  
    statement2
```

Statements 1 and 2 are processed with the iterator variable until the iterable is exhausted

```
«test_condition»  
  
stmt1      ↖  
  
    ↓      ↑  
  
stmt2      ↑  
  
    ↓      ↑  
  
stmt3      ↗
```

This control structure will be covered in much more details in a subsequent notebook.

Functions/Methods/Procedures/Sub

This provides a way to label a block of code. Thus

- You are able to access the implemented logic by invoking the name of the method.
- You write less code because now you are able to reuse existing logic.
- Functions are able to take input (arguments) and are able to give (return) a value
- Functions can call other functions or even itself (recursive)

- Functions can be accumulated into libraries to promote further reuse

Syntax

This will be covered after the next two chapters

```
def «function_name»(«inputs»):  
  
    statement1  
  
    statement2
```

Statements 1 and 2 are processed with the function is activated (invoked)

Functions will be covered in much more details in a subsequent notebook.

←↑→↓↖↗↘↙

Making Decision - if statement

Contents:

1. [Boolean expressions](#)
 - Comparison operators
2. [If statement](#)
 - Simple if statement without the else block
 - If statement with else block
 - Nested form
3. [Creating more complex logic](#) with `and` and `or`
4. [Summary](#)

Boolean expression

A boolean expression is a statement that evaluates to `True` or `False`.

A very common way to obtaining a boolean expression is by using comparison (relational) operators.

Comparison Operators

These operators takes two operands that are often of the same data types and returns a boolean value.

They are:

- `<` - evaluates to True only if the left side value is strictly less than the right side.
- `>` - evaluates to True only if the left side value is strictly greater than the right side.
- `==` - evaluates to True only if the left side value is the same as the right side.
- `!=` - evaluates to True only if the left side value is not the same as the right side.
- `<=` - evaluates to True only if the left side value is less or equal to the right side.
- `>=` - evaluates to True only if the left side value is more than or equal to the right side.

```
# we are defining two lists that will contain different types of elements  
# and then we will write code to show the results of the comparison between  
them
```

```
a = [-2, 1, 3, 1.2, 3.1, 9.8, False, True, True, 'apple', 'zebra',  
     'baby', [4, 9], [2, 3], [5]]  
b = [ 3, 1, 2, 1.4, 3.1, 7.2, True, True, False, 'zebra', 'apple',  
     'baby', [5, 0], [2, 3], [4]]
```

```
# ignore the logic of the code and just focus on the output  
# you will be able to understand the code after we cover  
# looping and iterables in future exercises
```

```
for i in range(len(a)):  
    e = f'{a[i]} < {b[i]}'  
    print(f'{e} -> {eval(e)}')
```

```
-2 < 3 -> True  
1 < 1 -> False  
3 < 2 -> False  
1.2 < 1.4 -> True  
3.1 < 3.1 -> False  
9.8 < 7.2 -> False  
False < True -> True  
True < True -> False  
True < False -> False  
'apple' < 'zebra' -> True  
'zebra' < 'apple' -> False  
'baby' < 'baby' -> False  
[4, 9] < [5, 0] -> True  
[2, 3] < [2, 3] -> False  
[5] < [4] -> False
```

```
#again ignore the logic of the code and just focus on the output
```

```
for i in range(len(a)):  
    e = f'{a[i]} == {b[i]}'  
    print(f'{e} -> {eval(e)}')
```

```
-2 == 3 -> False  
1 == 1 -> True  
3 == 2 -> False  
1.2 == 1.4 -> False  
3.1 == 3.1 -> True  
9.8 == 7.2 -> False  
False == True -> False  
True == True -> True  
True == False -> False  
'apple' == 'zebra' -> False
```

```
'zebra' == 'apple' -> False
'baby' == 'baby' -> True
[4, 9] == [5, 0] -> False
[2, 3] == [2, 3] -> True
[5] == [4] -> False
#again ignore the logic of the code and just focus on the output
```

```
for i in range(len(a)):
    e = f'{a[i]} >= {b[i]}'
    print(f'{e} -> {eval(e)}')
-2 >= 3 -> False
1 >= 1 -> True
3 >= 2 -> True
1.2 >= 1.4 -> False
3.1 >= 3.1 -> True
9.8 >= 7.2 -> True
False >= True -> False
True >= True -> True
True >= False -> True
'apple' >= 'zebra' -> False
'zebra' >= 'apple' -> True
'baby' >= 'baby' -> True
[4, 9] >= [5, 0] -> False
[2, 3] >= [2, 3] -> True
[5] >= [4] -> True
#again ignore the logic of the code and just focus on the output
```

```
for i in range(len(a)):
    e = f'{a[i]} <= {b[i]}'
    print(f'{e} -> {eval(e)}')
-2 <= 3 -> True
1 <= 1 -> True
3 <= 2 -> False
1.2 <= 1.4 -> True
3.1 <= 3.1 -> True
9.8 <= 7.2 -> False
False <= True -> True
True <= True -> True
True <= False -> False
'apple' <= 'zebra' -> True
'zebra' <= 'apple' -> False
'baby' <= 'baby' -> True
[4, 9] <= [5, 0] -> True
[2, 3] <= [2, 3] -> True
[5] <= [4] -> False
#again ignore the logic of the code and just focus on the output
```

```
for i in range(len(a)):
    e = f'{a[i]} != {b[i]}'
    print(f'{e} -> {eval(e)}')
-2 != 3 -> True
1 != 1 -> False
3 != 2 -> True
1.2 != 1.4 -> True
3.1 != 3.1 -> False
```

```
9.8 != 7.2 -> True
False != True -> True
True != True -> False
True != False -> True
'apple' != 'zebra' -> True
'zebra' != 'apple' -> True
'baby' != 'baby' -> False
[4, 9] != [5, 0] -> True
[2, 3] != [2, 3] -> False
[5] != [4] -> True
```

The `if` statement

The `if` statement allows you to run or skip a block of code based on whether a condition is `True` or `False`. is the simplest form of the if-statement.

The structure consists of:

- the keyword `if`
- an expression that evaluates to `True` or `False`
- a colon (`:`)
- an indented block of code

If the expression is `True`, the block is processed.

If it is `False`, the block is skipped.

Other languages use curly braces `{ }` to mark blocks. Python uses **indentation**

Syntax

Just a `True` block

```
if boolean expression:
    statement_1      # statement_1 is ONLY processed
                    # if the expression evaluates to True
```

Example:

```
#notice that the true block contains two statements and there are both
indented
score = 92
if score >= 90:
    print('Good job!')
    print(f'A score of {score} will get you an A+ grade')
Good job!
A score of 92 will get you an A+ grade
```

Example (condition is `False` no nothing prints):

```
# what does the following code output?
score = 80
```



```
if score >= 90:
    print(f'A score of {score} will get you an A+ grade')
```

Example with a line outside the block:

```
# in this example the last statement is not part of the if-statement
# so regardless of the results of the condition expression
# it will be processed
score = 90
if score >= 90:
    print(f'A score of {score} will get you an A+ grade')
print('This statement is not part of the if-statement')
```

```
# change the value of score to 95
# and run the code again
A score of 90 will get you an A+ grade
This statement is not part of the if-statement
```

If-Else statement

This version allows you perform **one of two** actions. i.e. code is able to complete either of two tasks.

Structure:

- `if` with an assertion terminated by a colon
- a block of code statements (True block)
- `else` followed by a colon
- a block of code statements (False block)

Only **one** block is ever executed in one pass

Again to identify the blocks to the interpreter, we indent all the statements in the blocks.

Syntax

```
if boolean_expression:
    # processed if boolean_expression is True
else:
    # processed if boolean_expression is False
```

Example:

```
# if statement with both True and False blocks
sales = 900
if sales > 1000:
    print(f'For sales of ${sales:,.2f} commission will be ${(sales - 1000) * 0.1:,.2f}')
else:
    print(f'For sales of ${sales:,.2f} commission will be $0.00')
```

```
#ignore the strange formatting code after the numbers, we will cover that
when we do loops
```

For sales of \$900.00 commission will be \$0.00

With different data:

```
# same example as above but the new value for sales will force the execution
of the else block
sales = 1500
if sales > 1000:
    print(f'For sales of ${sales:,.2f} commission will be ${(sales - 1000) *
0.1:,.2f}')
else:
    print(f'For sales of ${sales:,.2f} commission will be $0.00')
For sales of $1,500.00 commission will be $50.00
```

Nested If Statement

Nested forms involve multiple conditions occurring inside one another. They can appear inside either the True block or the False block.

Example form:

```
if expr1:
    statements1      # processed if expr1 is True
else
    if expr2:
        statements2 # processed if expr1 is False and expr2 is True
    else
        statements3 # processed if expr1 and expr2 are False
```

A clearer version (atleast in my opinion):

```
if expr1:
    statements1
elif expr2:
    statements2
else
    statements3
```

Another nested if statement

```
if expr1:
    statements      # processed if expr1 is True
    if expr2:
        statements # processed if both expr1 and expr2 are True
    else
        statements # processed if expr1 is True and expr2 is False
else
    statements      # processed if expr1 is False
```

A nested example:

```
# nested if statement
temperature = 14          #change the value to 15, 20, 25 and 30 and
execute the code
```

```

if temperature < 15:
    print(f'Cold')
elif temperature > 25:
    print(f'Hot')
else:
    print(f'Comfortable')
Cold

```

A more complex grading example:

```

#More complex nested if statement
score = 81                                #change the score and see if you can deduce
the grade
if score < 50:
    print(f'A score of {score} will get you an F grade')
elif score < 60:
    print(f'A score of {score} will get you an D grade')
elif score < 70:
    print(f'A score of {score} will get you an C grade')
elif score < 80:
    print(f'A score of {score} will get you an B grade')
else:
    print(f'A score of {score} will get you an A grade')
A score of 81 will get you an A grade

```

Comparisons with non-numbers

Strings (lexicographical order)

```

# Lexicographical comparison ( string )
print("alice < bob -> ", end='')
print("alice" < "bob")

print("alice > bob -> ", end='')
print("alice" > "bob")
alice < bob -> True
alice > bob -> False

```

List/tuples (element-wise comparison)

```

# Lists/Tuples (element-wise comparison, stops at first difference):
a = [0, 1, 2]
b = [0, 1, 2]
e = f'{a} == {b}'
print(f'{e} -> {eval(e)}')

b = [1, 2]
e = f'{a} == {b}'
print(f'{e} -> {eval(e)}')

e = f'{a} >= {b}'
print(f'{e} -> {eval(e)}')

e = f'{a} <= {b}'
print(f'{e} -> {eval(e)}')

```

```
e = f'{a} != {b}'
print(f'{e} -> {eval(e)}')
[0, 1, 2] == [0, 1, 2] -> True
[0, 1, 2] == [1, 2] -> False
[0, 1, 2] >= [1, 2] -> False
[0, 1, 2] <= [1, 2] -> True
[0, 1, 2] != [1, 2] -> True
```

You may also chain these comparison operators

```
x = 10
print(f'{ 1 < x < 10 }')
```

Creating more complex boolean expressions

Boolean expressions may be combined using the boolean operators `and` and `or`

<<boolean expression 1>> `and` <<boolean expression 2>> #return True only if both are True

<<boolean expression 1>> `or` <<boolean expression 2>> #return False only if both are False

```
age = 50
service = 6
```

```
if (age > 65) or (service > 10):           # the parenthesis are just for
    clarity, it is not necessary           # and is not normally present in
code
```

```
    print("You are eligible for the pension plan")
else:
    print("You are not eligible for the pension plan")
You are not eligible for the pension plan
age = 20
province = "QC"
```

```
if age >= 18 and province == "ON":
    print("You are eligible to vote in Ontario")
else:
    print("You are not eligible to vote in Ontario")
You are not eligible to vote in Ontario
age < 65 and service < 10
True
```

Truthy and Falsy Values

Python treats common values as True and False e.g.

True -> non zero number, non-empty sequence or mapping

False => zero, empty sequence or mapping

```
# do not try to understand the code, just focus on making sense of the output
for ex in [0, 1, 0.0, 1.3, '', 'Hello', set(), {1, 2}, (), (1, 4), [], [4],
           {}, {80: 'http'}, True, False, None]:
    print(f'{str(ex):<15} -> {str(type(ex)):<20} {bool(ex)}')
```

0	-> <class 'int'>	False
1	-> <class 'int'>	True
0.0	-> <class 'float'>	False
1.3	-> <class 'float'>	True
	-> <class 'str'>	False
Hello	-> <class 'str'>	True
set()	-> <class 'set'>	False
{1, 2}	-> <class 'set'>	True
()	-> <class 'tuple'>	False
(1, 4)	-> <class 'tuple'>	True
[]	-> <class 'list'>	False
[4]	-> <class 'list'>	True
{}	-> <class 'dict'>	False
{80: 'http'}	-> <class 'dict'>	True
True	-> <class 'bool'>	True
False	-> <class 'bool'>	False
None	-> <class 'NoneType'>	False

Summary

- if statements control the flow.
- Variants: if, if-else, if-elif-else, nested if.
- Conditions built from boolean expressions, comparison operators, and logical operators.
- Python uses **indentation** instead of braces for code blocks.
- Many values have inherent truthiness.

Making Decision - match statement

In its basic form the match statement is similar to the switch statement in most other languages. However, it is much more powerful than the version in C or Java because it does pattern matching. The main advantage is that it is more readable than the corresponding if statement.

Syntax

Simple match

```
switch «variable/expression»:
    case «pattern1»:
        statement1
    case «pattern2»:
        statement2
    case _:
```

statement3

Simple example - matching literals

```
# a simple example of matching literals  
command = "start"
```

```
match command:  
    case "start":  
        print("Starting...")  
    case "stop":  
        print("Stopping...")  
    case _:  
        print("Unknown command")
```

```
# change the value of command to test different cases
```

Example - matching literals

```
# an example  
# matching multiple values  
day = 'Sat' # notice thh choice of casing for day  
second = 7  
operator = '-'
```

```
match(day.lower()): #the lower() function returns all the  
characters in the lowercase  
    case 'sat' | 'sun': #two cases combined using the | operator  
        print(f'{day} is a family day')  
    case _:  
        print(f'{day} is a work day')
```

Example - variable binding

```
# this example illustrates variable binding  
point = (3, 4)
```

```
match point:  
    case (0, 0):  
        print("Origin")  
    case (x, 0):  
        print(f"On the X-axis at {x}")  
    case (0, y):  
        print(f"On the Y-axis at {y}")  
    case (x, y):  
        print(f"Point at x={x}, y={y}")
```

Example - guard condition

```
# this example illustrates guards conditions  
num = 15
```

```
match num:  
    case x if x < 0:  
        print("Negative")  
    case x if x % 2 == 0:  
        print("Even")
```

```

    case x:
        print("Odd")

```

Example - matching more complex patterns

```

# a slightly more complex example
x = int(input('Enter a number between 1 and 20: '))

```

```

match x:
    case 1:
        print('one')
    case 2 | 3:
        print('two or three')
    case x if x % 5 == 0:
        print('multiple of 5')
    case x if x > 16:
        print('greater than 16')
    case _:
        print('Value not in range')

```

Example - matching sequences

```

# a more complex example
# matching sequences
x = 'narendra Kowshal Drupatti Pershad'.split()
match x:
    case [x, y]:
        print(f'two items {x} {y}')
    case [x, y, z]:
        print(f'three items {x} {y} {z}')
    case [x, *y, z]:
        print(f'more than three items {x} {' '.join(y)} {z}')
    case _:
        print('unkown error')

```

Example - matching dictionary

```

# matching dictionaries
person = {'name': 'Narendra', 'age': 30, 'city': 'Delhi'} #matches the
first case
# person = {'name': 'Narendra', 'city': 'Delhi'} #matches the
second case
# person = {'name': 'Narendra'} #matches the
default case
match person:
    case {'name': name, 'age': age}:
        print(f'Name: {name}, Age: {age}')
    case {'name': name, 'city': city}:
        print(f'Name: {name}, City: {city}')
    case _:
        print('Unknown person data')

```

Example - matching classes

```

# matching classes
class Point:

```

```

def __init__(self, x, y):
    self.x = x
    self.y = y

p = Point(1, 2)

match p:
    case Point(x=0, y=0):
        print('Origin')
    case Point(x, y):
        print(f'Point at {x}, {y}')

```

Summary

- match is like switch, but much more powerful.
- Can match constants, variables, sequences, dicts, classes.
- Use | for multiple options.
- Supports guards (if) for extra conditions.
- Introduced in Python 3.10 so it won't work in earlier versions.
- The match statement checks patterns in order - first match wins
- Unlike C-style switch statements, Python's match doesn't have fall-through
- Use case _ as default.
- The _ pattern must come last if used
- The statement is exhaustive - your patterns should cover all possibilities or include a wildcard

Patterns can capture variables, but be careful about name conflicts